FedEXT: Differential Federated Learning with Complementary Extension of Edge Models

Zhenyu Wang
University of North Carolina at Chapel Hill
Chapel Hill, NC, USA
zywang@cs.unc.edu

Shahriar Nirjon
University of North Carolina at Chapel Hill
Chapel Hill, NC, USA
nirjon@cs.unc.edu

IEEE INFOCOM 2025 - IEEE Conference on Computer Communications | 979-8-3315-4305-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/INFOCOM55648.2025.11044645

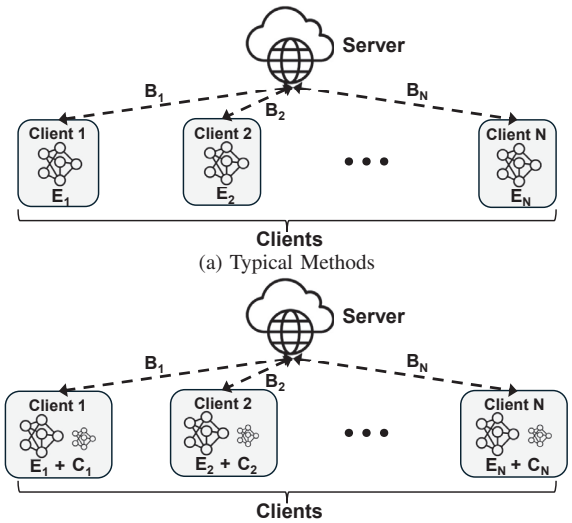
***Abstract*—We introduce FedEXT, an innovative federated learning framework designed to address the challenges of platforms with limited communication and computational resources. FedEXT employs a unique factorization-based methodology complemented by a paired neural network extension to improve model transmission and local training efficiency. This approach enables FedEXT to achieve the accuracy of larger models, which would otherwise be infeasible for federated learning on edge devices with stringent communication budgets. A key component of FedEXT is its periodic scheduler, which works in conjunction with local and global controllers to effectively manage model transmission. This scheduler uses a distinct learning and aggregation strategy to update the model parameters on local edge devices and the cloud server. The effectiveness of the framework is validated in both dataset-driven and real-world scenarios, demonstrating significant efficacy. FedEXT achieves improvements in accuracy ranging from 16% to 22% and a reduction in energy consumption by 29% to 34% compared to several state-of-the-art methods.**

***Index Terms*—Federated Learning, Complementary Neural Network**

I. Introduction

In recent years, driven by heightened privacy concerns, direct access to user data harvested by embedded and mobile devices from the cloud has become both undesirable and prohibitive [1]. In response to this pressing issue, federated learning [2]–[8] has surged to the forefront as a collaborative learning paradigm, harnessing the power of multiple clients to train a deep neural network. This approach fosters robust and effective learning by focusing on the transmission of local model parameters, thus expertly avoiding the exposure and transmission of sensitive data between the edge and the cloud. Through the aggregation of parameters from locally trained models in an iterative process, this distributed learning framework ensures strong privacy capabilities while simultaneously delivering superior model performance at the edge.

Unfortunately, achieving practical implementation of federated learning systems poses significant challenges in terms of in-network communication efficiency, local training efficiency, and overall model accuracy. These optimization goals often conflict, making the development of federated learning systems difficult, especially in resource-constrained settings. With limited transmission bandwidth and computational power at the edge, choosing the right model and designing an effective communication scheme are critical decisions.



(a) Typical Methods

(b) Our Approach

Fig. 1: Traditional approaches use less-accurate *edge* models E_i for training efficiency and to meet communication budgets. FedEXT combines *edge* E_i and *complement* C_i models, achieving higher accuracy and training efficiency while staying within budget B_i.

Recent research [9]–[11] has introduced innovative methods to improve federated learning efficiency by transmitting and training subgraphs or partial layers of smaller models. However, a major bottleneck remains: achieving both high global accuracy and high local training efficiency simultaneously. This difficulty is mainly due to the inflexibility and overly specialized nature of these approaches [9], [11], [12], along with additional hardware-software requirements [13].

To achieve superior global accuracy, one might consider using high-accuracy models. However, these models are typically larger and less computationally efficient, exceeding the transmission bandwidth and computational capacities of edge devices. This results in increased transmission costs and local training inefficiency. Thus, a new design paradigm is needed to balance these conflicting optimization goals effectively.

In this paper, we introduce FedEXT, a new federated learning paradigm that achieves high global accuracy by incorporating a *complement* network into the *edge* model. The *complement* network is designed to complement the *edge* model's feature representation while remaining compact and efficient for transmission and training. During inference,

Authorized licensed use limited to: Chung-ang Univ. Downloaded on January 21,2026 at 03:29:09 UTC from IEEE Xplore. Restrictions apply.

the combined network (*edge* and *complement*) achieves the accuracy of a state-of-the-art high-accuracy model. During training, intelligent scheduling of *edge* and *complement* model transmissions ensures training efficiency and adherence to communication budgets (e.g., energy cost). Figure 1 illustrates how the FedEXT model duo (*edge* and *complement*) meets the accuracy, training efficiency, and transmission budget goals. Due to communication and training budget constraints, traditional methods must rely on smaller edge models, which results in lower accuracy.

The construction of FedEXT presents two major challenges in achieving the desired overhead efficiency and global accuracy performance. First, a strategic approach is needed to guide the timing for transmitting *edge* and *complement* models. This method should consider the learning progress of both models to achieve high global accuracy during model aggregation on the server side while maintaining communication consumption at an acceptable level. Second, a specific training policy is required to handle the fusion of the complementary extension module and the *edge* model. The *complement* network fills in the missing features that the *edge* model does not capture in an efficient and compact form. This relationship between the *edge* and *complement* models is maintained throughout the local training process until the end of the learning phase.

To address these challenges, we propose an adaptive dynamic scheduler and a specialized training policy. First, to tackle the challenge of efficient model transmission, we introduce an adaptive scheduler that generates a cycle parameter to transmit the *edge* model based on the learning progress of the *edge* and *complement* models. The scheduler determines whether the server should send both models or only the *complement* model to the selected clients, with local training behavior adjusted accordingly. This dynamic adjustment ensures a manageable and efficient design as the models evolve.

Second, to handle the fusion of the complementary extension module and the *edge* model, we employ an instance-wise negative correlation term to preserve their relationship, ensuring that the *complement* model effectively complements the *edge* model's feature representation capability. The objective function includes a regression term between the fused output and the ground truth and a regularization term to force the *complement* model to focus on the relevant features. Additionally, we ensure flexibility in transmitting and aggregating *complement* models of varying sizes to accommodate each client's transmission constraints.

To demonstrate the efficacy of FedEXT, we conduct comprehensive evaluations in both dataset-driven and real-world deployment scenarios. The dataset-driven evaluation involves multiple datasets and state-of-the-art baselines, encompassing more than 106,000 images and 18 federated learning settings. Additionally, real-world experiments are conducted with 10 participants using over 1,000 user-collected images to assess FedEXT's performance in deployment settings. The evaluation shows that FedEXT achieves 16%-22% higher accuracy and 29%-34% lower energy consumption compared to multiple state-of-the-art approaches.

(a) Typical Methods (b) Our Approach

Fig. 2: Transmission models used in different federated learning methods.

Method	Commu-nication Efficiency	Model Hetero-geneity	Local Training Efficiency	High Global Accuracy
FedAvg [1]	✗	✗	✗	✗
pFedAtt [14]	✗	✗	✗	✗
pFedMe [15]	✗	✗	✗	✗
LG-FedAvg [11]	✓	✗	✗	✗
Hermes [9]	✓	✓	✓	✗
FedMask [16]	✓	✓	✓	✗
FedDST [10]	✓	✗	✗	✓
FedEXT (Ours)	✓	✓	✓	✓

TABLE I: Framework Comparison.

II. Motivation

A. Feasibility Analysis

Federated learning, a scalable and privacy-preserving approach to model training, has garnered significant attention in recent years [17]–[23]. While the method is well-suited for collaborative learning across multiple *client* devices, it poses significant challenges for resource-constrained edge devices. In such environments, the periodic exchange of model parameters incurs substantial time and energy overhead, impacting the overall efficiency and practicality of the system. Common practice [1], [14], [15] in these scenarios is to use low-overhead, and thus relatively less accurate, weak models at the edge devices to mitigate these issues.

We propose a new paradigm called *differential* federated learning to address these challenges. Instead of relying on a single model, we use a pair of complementary models (*edge* and its *complement*) that together mimic the accuracy of a larger, more accurate model while keeping the training and transmission overhead low. The *complement* network, being compact and complementary in its capabilities, enables clients to send this smaller network for most communication rounds and occasionally send the full network when necessary. This strategy significantly reduces the overall cost by carefully scheduling the transmission of the two lower-overhead models, thus optimizing the amortized cost of learning.

We can derive the condition for differential federated learning to be more cost-effective than state-of-the-art approaches by controlling the fraction of time, $0 < f < 1$, when only the *complement* model is sent to the server. As shown in Figure 2, we assume the costs of transmitting the *complement* and *edge* models are c and e, respectively. Additionally, let s be the size of the *edge* network portion that is sent and received by the state-of-the-art. The condition is formulated as follows:

$$\begin{aligned} &f \times c + (1-f) \times (c+e) < s \times 1 \\ &0 < \left(1 - \frac{s-c}{e}\right) < f < 1 \end{aligned} \tag{1}$$

Authorized licensed use limited to: Chung-ang Univ. Downloaded on January 21,2026 at 03:29:09 UTC from IEEE Xplore. Restrictions apply.

For example, in a federated learning approach that exchanges an average of 50% of the parameters compared to the original *edge* model (i.e. $s = 0.5e$) in each communication round, we can find a feasible f as long as the *complement* network is less than 50% of the combined size of *edge* and *complement*. Specifically, if the *edge* model is 10MB, then $s = 0.5e = 5$MB for a typical transmission approach. By choosing a *complement* model with a size of 1MB, we achieve a 30% communication cost reduction with $f = 0.75$.

B. Qualitative Comparison with State-of-the-art

Federated learning methodologies designed for edge computing typically aim to optimize communication efficiency, model heterogeneity, local training efficiency, and global accuracy. In Table I, we compare our proposed FedEXT with state-of-the-art federated learning baselines across these key properties of efficient federated learning. Among these, the following three baselines are the closest compared to FedEXT.

- **FedAvg** [1]: The classical federated learning framework performs learning iteratively by conducting local training on client devices and then transmitting the results to the server. The server simply averages the collected weights and re-distributes them back to each client.
- **Hermes** [9]: A federated learning framework for personalization involves performing local *structured* pruning during each communication round under certain conditions. The methodology identifies a subnetwork, which is then transmitted to the server for aggregation. This method significantly improves communication and local training efficiency while providing a desired personalization performance.
- **FedDST** [10]: The state-of-the-art federated learning framework employs dynamic sparse training with *unstructured* pruning to improve client model sparsity. It introduces strategic transmission for the pruned subnetwork and mask. However, its computational efficiency is limited by hardware and software constraints due to the usage of unstructured pruning technique.

FedAvg fails to be effective in all four key properties. FedDST faces challenges in flexible model heterogeneity and training efficiency, and Hermes fails to achieve high global accuracy. Our proposed FedEXT excels in all four properties, as assessed in both dataset-driven evaluations and real-world deployment experiments (details are described in Section V and Section VI).

III. FedEXT Design

We describe a new factorization-based federated learning approach — FedEXT that uses a pair of low-overhead models (*edge* and its *complement*) to significantly lower the cost of communication and local training while increasing the global accuracy performance.

A. Overview

In typical federated learning, two or more client devices train their models locally on private datasets. These client devices then upload their model parameters to a central server for aggregation. After the server aggregates the parameters, the updated model is downloaded by the clients for further training. This process is repeated for several rounds until the model converges. FedEXT follows a similar process with a notable difference. In FedEXT, each client operates a fusion of two complementary models (*edge* and *complement*). During each round, only the *complement* model is sent to the central server, while the *edge* model is transmitted at a much lower frequency. This approach leverages the complementary strengths of the two models to achieve higher global accuracy while optimizing communication efficiency by heuristically finding the optimal frequency of *edge* model transmissions. This frequency scheduling not only guides the model transmission but also manages the local training of *complement* and *edge* models. As a result, the efficiency of local training computations is also improved. Moreover, FedEXT considers the communication and computation constraints of each client device to accommodate model heterogeneity. We also use a mask-aware model aggregation method to handle the varying sizes of weight parameters across different client devices. This design offers substantial flexibility and scalability, enabling it to effectively handle model heterogeneity.

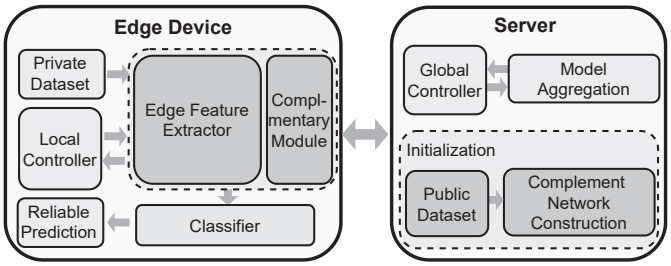


Fig. 3: Design of FedEXT.

B. System Design

The entire system consists of multiple components cooperating together to achieve a high global accuracy performance as well as low communication and local training energy consumption. The system initialization, model aggregation, and global control are executed on the server side, while the local control and training are performed on each client device. Figure 3 illustrates the interactions between a client and the server in FedEXT. There are four major steps:

- **Step 1 – Initialization:** At first, a light-weight *edge* model, and a computationally heavy but relatively more accurate *base* model are chosen or created, e.g., by utilizing pre-trained models [24] for resource-constrained systems, or by training such models from scratch. A *complement* model is constructed at the server using *base* and *edge* models, as well as a *public* dataset based on the given communication budget. The fusion of *edge* and the *complement* models — which achieve equivalent feature representation capability to the *base* model when fused together — are pushed to all clients. The *base* model is no longer required after this step.
- **Step 2 – Local Training.** Each client uses its own *private* dataset to locally train the *complement* model as well as

Authorized licensed use limited to: Chung-ang Univ. Downloaded on January 21,2026 at 03:29:09 UTC from IEEE Xplore. Restrictions apply.

a portion of the *edge* model that extracts features. The *complement* model is trained in every single communication round; however, the training process of the *edge* model is synchronized with the transmission scheduling strategy handled by the local controller of each client device. To achieve a proper learning behavior, the weight parameter updates follow a specially designed objective function that leverages their complementary properties. The detailed training process is presented in Section IV-B.

- **Step 3 – Communication.** When local training ends, the loss values of the *complement* and the *edge* networks are sent to the server. At the server, these values are summed and compared by a controller to determine whether to instruct the client to train only the *complement* model or both (*edge* and *complement*) using the following criteria: $\frac{\Delta\mathcal{L}_C}{\Delta\mathcal{L}_E} > C_{thresh}$, where, $\Delta\mathcal{L}_E$ and $\Delta\mathcal{L}_C$ denote the percentage drop in loss since the *edge* and the *complement* models were sent to the server, and C_{thresh} is a constant that controls how frequently the *edge* model is sent along with the *complement*. As such, an adaptive periodic communication scheduler is constructed based on the learning progress of *edge* and *complement* model. More details are presented in Section IV-C.
- **Step 4 – Aggregation.** The server aggregates the *edge* models by taking an weighted average following standard federated learning practices [1]. Since the architecture of *complement* may vary across clients due to different overhead constraints, their neurons are averaged by the overlapping frequency based on their masks. This aggregation technique ensures that the model preserves its learned knowledge while allowing for the manageable merging of these updated weights, even when they vary in size. In the end, the aggregated models are distributed to the participating client devices.

IV. Construction of FedEXT

In this section, we present the detailed steps of FedEXT construction, including system initialization, local model training, communication scheduler construction, and model aggregation. These methodologies form a cohesive framework for implementing FedEXT, balancing high accuracy with efficient resource utilization.

A. System Initialization

Overview. Proper initialization sets a solid foundation for further communication scheduling and model learning to achieve high global accuracy in FedEXT. At the heart of this process is the construction of a compact *complement* network module, designed to support the *edge* model's predictions while minimizing communication and computation resource consumption during the iterative cooperative learning. By using a carefully designed policy to complement the feature representation of the *edge* model, the fusion of *complement* and *edge* model ensures efficient and effective prediction performance.

Design Proposition. The server hosts a public dataset used for constructing the *complement* model before distributing it to client devices. Both the *base* model and the *edge* model are pretrained on this public dataset for initialization. The *complement* model is designed with the specific capability to learn and capture the critical features that the initialized *edge* model misses but are successfully identified by the high-accuracy *base* model. This feature learning is accomplished in a highly compact form, making the *complement* model lightweight to meet the communication budgets of model transmission. To achieve this, the *complement* model leverages a state-of-the-art structured subgraph extraction technique [25], which enables the *complement* model to partially distill the essential feature representation capabilities of the *base* model into a much smaller and computationally efficient form with adjustable sparsity for each layer. This design strategy ensures that the *complement* model enhances the prediction performance of the *edge* model without imposing heavy computational demands, such that its lightweight nature is particularly beneficial in federated learning settings with constrained transmission budgets.

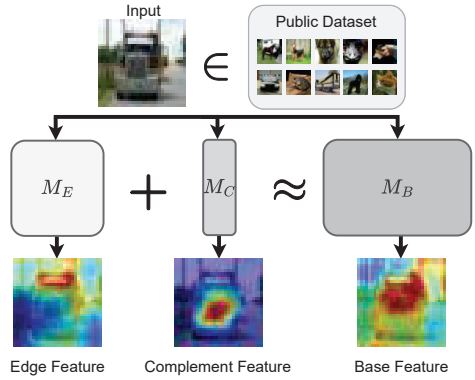


Fig. 4: The constructed *complement* network M_C complements the feature representation of the *edge* model M_E to achieve a similar feature representation as the *base* model M_B when they are fused (*complement* + *edge*).

Complement Network Construction. The mask used for extracting the architecture of subgraph from *base* model is created by finding the appropriate corresponding sparsity rate of each layer after applying the optimization framework, which utilizes the expanded graph derived from *base* model to learn the optimal layer-wise sparsity rate. This process is iteratively applied to create a lightweight subgraph backbone that preserves essential feature representation capability for complementing *edge* model. After determining the architecture of *complement* model, we construct the *complement* feature extractor, which fuses features extracted by both the *edge* model and the *base* model's subgraph to adhere to their complementary behavior. The *complement* network is explicitly fine-tuned by using the following objective function to ensure it complements the *edge* network's capability to represent input data:

$$\mathcal{L}_{complement} = \lambda_0\mathcal{L}_{MSE} + \lambda_1\mathcal{L}_{FNC} + \lambda_2\mathcal{L}_{SR} \qquad (2)$$

, where $\mathcal{L}_{MSE}$ is the Mean Square Error (MSE) loss function, ensuring that the quality of the fused feature (*edge* + *complement*) closely matches the *base* model's feature; $\mathcal{L}_{FNC}$

Authorized licensed use limited to: Chung-ang Univ. Downloaded on January 21,2026 at 03:29:09 UTC from IEEE Xplore. Restrictions apply.

is the feature-wise negative correlation term [25], [26], which enables complementarity between the *edge* and *complement* models; $\mathcal{L}_{SR}$ is the sparsity regularization term to make the *complement* network focus on the relevant features [27]. The hyperparamters λ_0, λ_1 and λ_2 control the relative strength of corresponding terms. During the fine-tuning of *complement* model, *edge* and *base* models are frozen, only the *complement* model's weight parameters are learnable. Noting that the fused model (*edge* + *complement*) has a lower computational cost and smaller parameter size compared to the *base* model, while achieving similar feature representation capability. In Figure 4, we present an example image contains a truck from a public CIFAR10 [28] dataset that is misclassified by the *edge* model M_E but correctly predicted by the *base* model M_B. The *complement* network, denoted as M_C, complements the missing features (i.e., the windshield, headlights and bumper of the truck) that the *edge* model M_E fails to capture. As a result, M_E's error is corrected after adding these missing features, achieving a similar feature representation capability as M_B when it is fused with M_C.

B. Local Model Training

Overview. In this section, we delve into the details of local training in FedEXT. The models distributed to each client device need proper learning guidance to adapt to the proposed federated learning framework. The updating policy of learnable parameters and objective functions are carefully designed and coordinated with the communication scheduler to achieve stable and synchronized behavior, while preserving the complementary behavior between the *edge* and *complement* models. This scheduling-based design enables a more flexible and efficient model training process, achieving high accuracy with limited communication and computation budgets.

Design Proposition. After initialization, both the *complement* and *edge* models are distributed to each client device, where they undergo training using the respective private datasets collected by each user. Unlike traditional federated learning approaches that update the *edge* model in every communication round, we introduce a scheduler-based methodology aimed at optimizing efficiency. This methodology updates corresponding model parameters only when necessary, effectively reducing communication costs and conserving local training energy (details of the scheduling process is presented in Section IV-C). A local controller guides the learning progress of the *edge* feature extractor and the *complement* network, enabling training either solely for the *complement* network or for both. The local training process is tightly integrated with the communication strategy between clients and server, ensuring precise tracking of parameter updates and cohesive integration of model learning and communication.

Objective of Local Training. To measure the learning progress of the *edge* feature extractor and the *complement* model, we create a lightweight measuring block for each model, using Multi-Layer Perceptron. This block computes the feature representation quality of the *edge* feature extractor and *complement* model using the l_2-norm between their output and ground truth as criterion. Specifically, the parameter updating process is formulated as below:

$$\begin{aligned} W_{T+1} := {} & W_T^{classifier} - \eta \nabla W_T^{classifier} + \\ & W_T^{complement} - \eta \nabla W_T^{complement} + \\ & \mathbb{1}_{LC}(W_T^{edge} - \eta \nabla W_T^{edge}) \end{aligned} \tag{3}$$

, where W_{T+1} is the learnable weight parameters updated at round $T+1$ in local device; W_T^{edge} and $W_T^{complement}$ are the weight parameters of *edge* feature extractor and *complement* model including their measuring blocks in round T, respectively; $W_T^{classifier}$ is the classifier portion that generates the final output by taking the fused feature from *edge* and *complement* model; η is the learning rate for updating the parameters; $\mathbb{1}_{LC}(\cdot)$ is the indicator function that is activated when signal from local controller is released. Unlike the *complement* model construction process, the *edge* model is learnable during the training of FedEXT, which is included inside the optimization scope for each client device accordingly. Noting that the size of weight parameters of *complement* network is not the same across different devices. Due to varying communication and device computation power constraints, we use different sizes of *complement* models to adapt to various types of devices and resource budget restrictions, incorporating model heterogeneity for FedEXT.

The objective function for local training is similar to Equation (2), however, the negative correlation term is adjusted as follows due to the lack of *base* model's feature representation during local training:

$$\mathcal{L}_{NC} = \lambda_a \mathcal{L}_{corr} + \lambda_b \mathcal{L}_C + \lambda_c \mathcal{L}_E \tag{4}$$

, where each loss term is presented in details as below

$$\mathcal{L}_{corr} = \frac{1}{N} \sum_{i=1}^{N} \left(\left(d_E^{(i)} - d_{E||C}^{(i)} \right) \left(d_C^{(i)} - d_{E||C}^{(i)} \right) \right) \tag{5}$$

$$\mathcal{L}_C = \frac{1}{N} \sum_{i=1}^{N} \left(d_C^{(i)} - d_{GT}^{(i)} \right)^2 \tag{6}$$

$$\mathcal{L}_E = \frac{1}{N} \sum_{i=1}^{N} \left(d_E^{(i)} - d_{GT}^{(i)} \right)^2 \tag{7}$$

The $d_C^{(i)}$ and $d_E^{(i)}$ is the respective output of the measuring blocks of *edge* feature extractor and *complement* network for the i-th ($1 \leq i \leq N$) local training example. $d_{E||C}^{(i)}$ is the fused output after combining the outputs of measuring blocks from the *edge* and *complement* models together. $d_{GT}^{(i)}$ is the ground truth of the i-th local training example collected by the user. λ_a, λ_b and λ_c represents the contribution of each loss term to negative correlation learning. Similarly, instead of measuring from the perspective of feature representation, the MSE term of Equation (2) is adjusted to instance-wise measurement using cross-entropy due to the lack of *base* model in the local edge device.

Authorized licensed use limited to: Chung-ang Univ. Downloaded on January 21,2026 at 03:29:09 UTC from IEEE Xplore. Restrictions apply.

C. Communication Scheduler

Overview. In this section, we introduce a strategic method for transmitting two models (*edge* and *complement*). Given the lightweight nature of the *complement* model in terms of parameter size and computational efficiency, our primary focus is to design an effective communication strategy for the *edge* model. This strategy aims to ensure that the *edge* model matches the learning progress of the *complement* model while maintaining relatively low resource consumption. It determines when to send both models or only the *complement* model. This strategy is presented in a scheduler form that is easy to integrate into the federated learning scheme, enabling a manageable design.

Design Proposition. To ensure the correct transmission arrangement, we propose a communication scheduler based on the learning progress of models running in the client devices. An adaptive transmission cycle $p[T]$ is defined, which evolves with respect to round T based on the current learning progress of *edge* and *complement* models. Briefly speaking, after determining the transmission cycle $p[T]$, *edge* model is transmitted in every $p[T]$ rounds, and *complement* model is transmitted in every single round, thanks to its lightweight size and low computational cost. The calculation of $p[T]$ is only activated after consuming the current idling budget, as such, at the end of $p[T]$ cycle period, *edge* model is updated once for both local training in client devices and server-client model transmission. Then $p[T]$ is re-computed again to prepare for the next cycle period.

Scheduler Construction. Our design goal is to enable a more flexible increment of $p[T]$ to optimize the communication costs and local training energy consumption, while keeping the decrease of $p[T]$ manageable and not over-reduced to avoid potential unnecessary resource allocation. To achieve this, we introduce a directional cycle updating unit for increasing the $p[T]$ when it is needed:

$$C_T = \frac{\Delta\mathcal{L}_C^1}{\mathcal{L}_C^{T-1}} \Big/ \frac{\Delta\mathcal{L}_E^{p[T]}}{\mathcal{L}_E^{T-p[T]}} \tag{8}$$

, where we have the following definitions:

$$\Delta\mathcal{L}_E^{p[T]} = |\mathcal{L}_E^T - \mathcal{L}_E^{T-p[T]}| \tag{9}$$

$$\Delta\mathcal{L}_C^1 = |\mathcal{L}_C^T - \mathcal{L}_C^{T-1}| \tag{10}$$

Here, L_E^T and L_C^T represent the summation of loss values of the *edge* and *complement* models at communication round T from all the selected clients, following Equations (6) and (7), respectively. When the increment unit C_T is greater than a predefined threshold C_{thresh}, the new period cycle is increased by $\phi \times C_T$, where ϕ is the step rate only for increasing the cycle $p[T]$. For decreasing the $p[T]$, since we have less tolerance for potential oscillation in reduction that leads to a much higher frequency for transmitting *edge* model, we use a fixed reduction unit C_R to achieve balanced and controllable cycle updates, avoiding potential over-allocation of resources for communication and local training. Similarly, a reduction step rate hyperparameter τ is provided for smooth updates.

By utilizing the above definitions and the introduced parameters, we construct a scheduler updating scheme for global controller, as such, when the current cycle budget is consumed exhaustively, we follow below formula to compute the next cycle period $p[T+1]$ to guide the subsequent model transmission and local training behavior:

$$p[T+1] = \begin{cases} p[T] + \phi \times C_T, & \text{if } C_T > C_{thresh} \\ p[T] - \tau \times C_R, & \text{otherwise} \end{cases} \tag{11}$$

In an extreme case, when L_E is not changing and becomes stable, training the *edge* model becomes less critical, making more frequent transmission and local training unnecessary. Specifically, when we have $\Delta L_E^{p[T]} \to 0$, the corresponding $C_T \to +\infty$, which leads to a fairly large $p[T]$. However, this may also occur during the middle of training due to the oscillation of the learning process. To avoid this potential inappropriate update, we set an upper bound for $p[T]$ to prevent the updating cycle from becoming too large.

D. Model Aggregation

Overview. In this section, we describe the mask-aware model aggregation technique to handle the model heterogeneity during the iterative training in FedEXT. As we have discussed in Section IV-A, a standard *complement* model is constructed during system initialization. This *complement* network can be further pruned based on the l_2-norm criterion with adjustable sparsity since a sparse regularization term is used during its fine-tuning. We assign different masks to each client model based on their communication and device computation constraints. These masks are applied to the standard *complement* network to generate specialized *complement* models for each client. A model aggregation approach is used to transform the various local models to a standard size without losing their preserved weight information.

Design Proposition. The models that transmitted to server needs an extra process before re-distributing them back to the clients. A common approach [1] is to compute the average weights across different models received from clients. This approach requires all models to have the same architecture to ensure the weight parameters have the same shape. This model aggregation approach is suitable for the unchanged *edge* model; however, it requires a more sophisticated design for the *complement* model, which can be adjusted in shape and size to accommodate frequent transmissions and varying communication budgets for a more flexible federated learning system. To handle this challenge, we use a mask-aware model aggregation technique similar to [9] that considers the overlapping frequency of each weight element when performing the aggregation on the *edge* and *complement* models.

Mask-Aware Aggregation. We define a function $g(W, M) = W \odot M$, which takes the weight parameters as input and applying the predefined structured mask M to generate sparse

Authorized licensed use limited to: Chung-ang Univ. Downloaded on January 21,2026 at 03:29:09 UTC from IEEE Xplore. Restrictions apply.

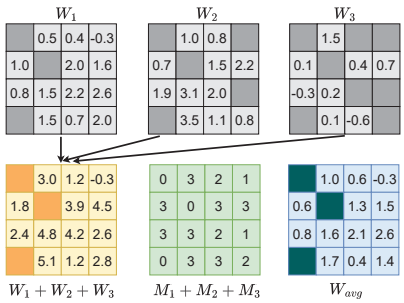


Fig. 5: Example of aggregating three weights with different masks: W_{avg} is obtained by summing the three sparse weight matrices — W_1, W_2, and W_3, dividing by the aggregated mask representing frequencies (M_1 + M_2 + M_3). For each weight W_i, the dark gray cell is the empty element after applying corresponding mask M_i to the original weight matrix.

weights for the corresponding model or layers using element-wise multiplication $\odot$. Noting that we use structured mask, which further allows us to reorganize the masked weight parameters such that the ultimate weights W' are represented in a dense matrix form but with smaller size and computational cost. We further construct another function $h(W', M) = W$, which converts the reorganized weight W' back to the sparse weight matrix W with its original shape based on the information from mask M. These functions ensure a smooth conversion between the various shapes of local *complement* model and a standard shape for *complement* model aggregation in the server without deteriorating the preserved information. The updating process for *complement* model is:

$$W_{avg}^{complement} := \frac{\sum_{i=1}^{K} h(W_i^{complement}, M_i^{complement})}{\epsilon + \sum_{i=1}^{K} M_i^{complement}} \tag{12}$$

$$W_i^{complement} := g(W_{avg}^{complement}, M_i^{complement}) \tag{13}$$

The received model weight parameters $W_i^{complement}$ from the i-th ($1 \leq i \leq K$) client are first transformed back to the pre-defined shape based on the mask $M_i^{complement}$. The computation of neurons for the weight $W_{avg}^{complement}$ involves calculating the overlapping frequencies based on the masks. Consequently, each neuron of the averaged weights retains the information from the corresponding multiple devices' private data distributions. We add ϵ, a small perturbation to denominator to avoid overflow during element-wise division. An illustration is shown in Figure 5, given three weight matrices, the averaged result is computed by aggregating both mask and weight parameters. In the end, the averaged weights are fed into $g(\cdot)$ to generate dense weights that have the corresponding shape with updated parameters. These newly generated weights are transmitted to each client and prepared for the next round. For the *edge* model, we follow the traditional way [1] to aggregate the models and re-distribute the models back to the clients. In an ideal scenario, all the client devices use the same *complement* model architecture, such that Equation (12) becomes a vanilla averaging operation.

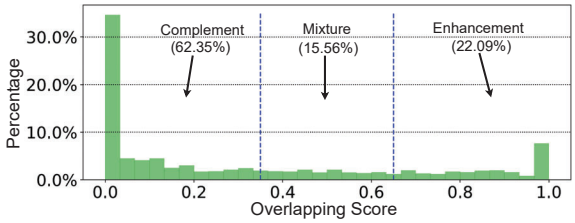
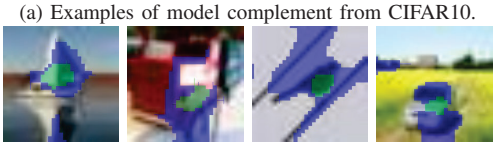


Fig. 6: Distribution of overlapping score between *complement* and *edge* models. They are separated into three different categories, with the attached percentage that accounts for each.

(a) Examples of model complement from CIFAR10.



(b) Examples of model enhancement from CIFAR10.

Fig. 7: Different types of *complement* model behavior from the VGG16-VGG8 pair.

V. Dataset-Driven Evaluation

We conduct dataset-driven experiments to compare FedEXT with state-of-the-art federated learning approaches.

A. *Construction of* Complement *Model*

We select VGG16 [29] as the *base* model and VGG8 [29] as the *edge* model, with parameter sizes of 15.25M and 4.44M, and FLOPS of 0.314G and 0.068G, respectively. The *complement* network is constructed by extracting 3.07% of parameters from the *base* model, which improves 9.45% of accuracy to *edge* model after the model fusion. We use CIFAR10 as our public dataset to construct the *complement* network. We present the overlapping score distribution of the *complement* model to understand its behavior in complementing the features extracted by the *edge* model. As shown in Figure 6, we plot the distribution of overlapping scores across the samples that are incorrectly predicted by the *edge* model solely but correctly predicted after adding the *complement* model. The overlapping score is computed by using the intersection region between the *complement* and *edge* models divided by the *complement* model's focused region [25]. There are two peaks in the figure where the overlapping score is close to 0 and close to 1. Instances with a higher overlapping score indicate that the *complement* model enhances the *edge* model by suppressing noisy regions and strengthening critical regions, thereby correcting the *edge* model's predictions. Instances with a lower overlapping score show that the *complement* model complements regions missed by the *edge* model, leading to mispredictions. By adding these missed features, the fused features are able to provide correct predictions. The overlapping scores that lie in the middle of the plot indicate a mixture behavior, which is a combination of enhancement and complement.

Authorized licensed use limited to: Chung-ang Univ. Downloaded on January 21,2026 at 03:29:09 UTC from IEEE Xplore. Restrictions apply.

Datasets	Total Images	Usage	Test Images
CIFAR10	60,000	Public	10,000
CIFAR100-S	6,000	Private	1,000
EuroSAT	27,000	Private	2,000

TABLE II: Datasets used in evaluation.

Figure 7 presents several examples to illustrate the complement and enhancement behaviors from the CIFAR-10 dataset. The blue color highlights the region that the *edge* model focuses on, while the green color highlights the *complement* model's activation region. The *complement* model complements the *edge* model's feature representation by capturing the facial features (e.g., eyes, nose, forehead) of different animals. Additionally, the *complement* model enhances the *edge* model's existing features by highlighting the valid regions of the target objects when the *edge* model includes the noisy and less critical parts (e.g., background) in its considerations.

B. Baseline Configuration

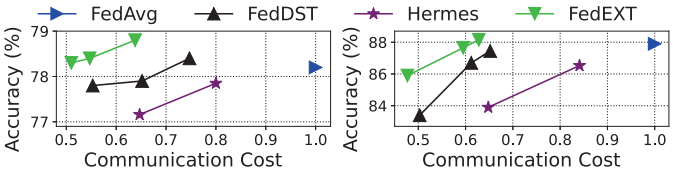
We simulate 100 clients with four communication budget constraints, each allocated to 25% of them. The simulation involves approximately 900 communication rounds to guarantee the convergence of the aggregated model at the server. 80 clients are used per communication round to update the model. To achieve non-IID data distribution, each client uses data from 3 out of 10 classes to assess the capability for handling data heterogeneity. The learning rate is 0.001, batch size is 10, local training epoch is 2.

We compare FedEXT against FedAvg [1] and two state-of-the-art approaches: Hermes [9] and FedDST [10]. We evaluate their performance in two scenarios: intra-domain tasks on similarly distributed public (CIFAR10 [28]) and private (CIFAR100-S [28]) datasets, and cross-domain tasks on differently distributed public (CIFAR10) and private (EuroSAT [30]) datasets. For each scenario, we show the accuracy and communication overhead of each federated learning technique. All baselines use the same training configuration as FedEXT for a fair comparison. Table II provides a summary of the datasets. CIFAR100-S denotes a 10-class subset of the original CIFAR100 [28].

C. Communication Cost and Accuracy

Figure 8 (a) and (b) presents a quantitative comparison of FedEXT with FedAvg, Hermes, and FedDST for intra-domain and cross-domain tasks. Since the accuracy and communication cost of these algorithms are related, we present their performance on accuracy vs. communication cost scatter plots. The communication costs are normalized with respect to FedAvg's.

FedDST employs an unstructured pruning-based mask to guide the subnetwork of the client model VGG8 but still does not achieve high efficiency as FedEXT on either scenario. Hermes employs different subnetworks in each client model for personalization; however, since the uncommon parts adapt to the local training data distribution, a biased tendency is observed in this case. The compactness of the given local client network also affects the subnetwork selection, which



(a) Intra-Domain Scenario (b) Cross-Domain Scenario

Fig. 8: Comparison with federated learning baselines.

Models	Param.(M)	FLOPS(M)	Accuracy(%)
MobileNetV1	7.18	99.18	78.47
VGG19	20.57	399.27	86.77
FedEXT	8.43	152.45	85.16

TABLE III: Models used in FedEXT deployment.

in turn provides different effects to the model aggregation and accuracy. As we observe from Figure 8, FedEXT provides the best trade-off between communication cost and accuracy. FedEXT can adjust both the scheduling behavior and the mask size of the *complement* model based on varying communication constraints. It achieves 78.4% and 85.9% accuracy in intra-domain and cross-domain tasks with 54.7% and 47.7% of communication cost, respectively.

VI. Deployment Experiment

In order to evaluate the performance of FedEXT in real-world scenarios, we conduct a week-long deployment experiment involving 10 participants who contribute pictures of their daily food intake using their smartphones. The goal is to train an image classifier that learns to classify foods in a federated learning setup.

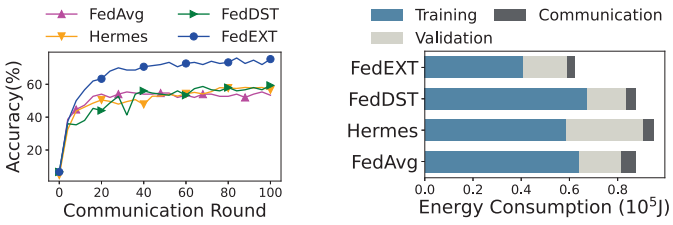
A. Models and Datasets

We use VGG19 [29] and MobileNet-V1 [31] with multiplier 1.5 as the *base* and the *edge* models, respectively. CINIC10 [32] is used as the *public* dataset, which contains 270,000 images from 10 different categories. Table III shows the size, FLOPS, and accuracy of these models along with the fused model between *edge* and *complement* network on CINIC10. The corresponding *complement* model is constructed offline using one GeForce RTX 2080 Ti with 32GB of RAM that takes roughly 158 hours to complete the task. Pictures taken by each participant constitute the *private* dataset for that client. Total 1,009 images are taken by the participants over the course of the experiment.

B. Baseline Configuration

Participants use a custom app on their Google Pixel 2XL phone which runs four federated learning algorithms (FedEXT, FedDST, Hermes, and FedAvg) in the background to train the local models (sequentially for each algorithm) on the smartphone and to upload/download model parameters to/from the server. Each device performs one epoch of local training with a learning rate of 0.001. The accuracy of the models is periodically validated on the phone using ground truth labels obtained from the participants, and the results are averaged. The total number of communication rounds is approximately

Authorized licensed use limited to: Chung-ang Univ. Downloaded on January 21,2026 at 03:29:09 UTC from IEEE Xplore. Restrictions apply.



(a) Accuracy curves of different baselines.

(b) Break down of energy consumption.

Fig. 9: Accuracy and energy consumption comparison with different federated learning baselines.

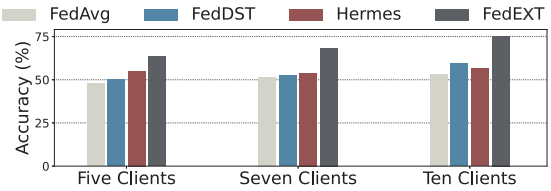


Fig. 10: Accuracy comparison with different baselines across various number of participants.

100 and the local batch size is 10. The target pruning threshold for Hermes is 0.3. The network sparsity for FedDST is 0.3.

C. Energy Consumption and Accuracy

We present the comparison of their accuracy and energy consumption among four baselines. Figure 9 (a) shows the accuracy of each baseline after every four communication rounds. FedDST and Hermes achieve 3% - 6% higher accuracy than FedAvg while maintaining higher model sparsity upon convergence. A potential reason for FedDST occasionally performing worse than Hermes is its lack of support for batch normalization layers, which are skipped during unstructured pruning on MobileNetV1 in FedDST. FedEXT outperforms all three baselines, achieving over 15% higher accuracy with the lowest system overhead as training converges.

Figure 9 (b) shows the energy consumption of each baseline for a randomly chosen client. The remaining clients show similar energy patterns. Energy is estimated by monitoring the battery percentage drop in a controlled environment, with the active percentage range constrained to avoid regions where the battery behavior exhibits notable nonlinearity. The total energy consumption is estimated from the battery capacity (calibrated using AccuBattery [33]), which is broken down into cost of local training, local validation, and communication. The energy spent on validation is sightly higher in FedEXT due to the additional *complement* network, but is still comparable to other baselines. Most of the energy is consumed during local training, which is a critical optimization goal in federated learning systems. FedEXT consumes the least energy for communication and local training among all four baselines.

Figure 10 presents the evaluation of different baselines with varying numbers of participants. As the number of clients involved in the training increases, FedEXT achieves higher accuracy due to the richer data distribution and the more comprehensive learned model parameters. By comparing it with other baselines, we observe that FedEXT consistently achieves the highest global accuracy. FedDST and Hermes achieve higher accuracy than FedAvg with much sparser models but fall short of FedEXT in all scenarios.

VII. Related Work

A. Distributed Deep Learning and Federated Learning

Distributed deep learning algorithms have flourished in recent years. Significant effort has been devoted to optimizing communication costs through various approaches, such as quantization [34]–[36] and sparsification [37], [38], to enable more efficient distributed model learning. As privacy and security needs for real-world deployment grow, federated learning has gained attention by keeping data decentralized on client devices. This type of distributed machine learning paradigm has been commercially deployed in real-world applications in recent years [39]. In this decentralized learning framework that protects user privacy, a server aggregates model parameters from clients instead of raw data. Models are trained locally on edge devices using private datasets. This aggregation of client models, trained on heterogeneous data distributions, creates a robust model capable of handling diverse data across devices.

B. Factorization-based Federated Learning

Many works have been proposed to achieve efficient federated learning, with one popular approach being model factorization. Federated learning methods based on model factorization, such as those discussed in [9]–[12], [40], transmit only a fraction of the local model to reduce the communication burden. Approaches that factorize models from a subnetwork perspective, such as those proposed in [9], [10], [40], require specific selection of the substructure of the transmission model to retain the desired local/global accuracy, and sometimes need peculiar software [13] to optimize the local computation cost in edge device due to the unstructured mask type. Moreover, a more straightforward layer-wise factorization methods [11], [12] provide less flexibility during the iterative federated training due to the restricted configuration. A similar attention-based architecture is proposed in [41]; however, their design is constrained to handle time-series tasks. In this paper, we propose FedEXT, inspired by the work presented in [25]. FedEXT excels in communication efficiency, local training efficiency, and global accuracy, while maintaining model heterogeneity without specific software configuration.

VIII. Conclusion

In this paper, we propose a novel model factorization-based federated learning method, FedEXT, which provides state-of-the-art efficiency in communication and local training while achieving high global accuracy. To achieve this, we propose a communication scheduler to guide the transmission of *edge* and additional complementary extension module to optimize the communication cost. We further introduce a learning policy for local and global model updates. Comprehensive evaluations against state-of-the-art baselines in both dataset-driven and real-world deployment experiments are conducted to demonstrate the superiority of the proposed FedEXT.

Authorized licensed use limited to: Chung-ang Univ. Downloaded on January 21,2026 at 03:29:09 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *AISTATS 2017*.

[2] Dongqi Cai, Yaozong Wu, Shangguang Wang, Felix Xiaozhu Lin, and Mengwei Xu. Efficient federated learning for modern nlp. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*, pages 1–16, 2023.

[3] Xiaomin Ouyang, Zhiyuan Xie, Heming Fu, Sitong Cheng, Li Pan, Neiwen Ling, Guoliang Xing, Jiayu Zhou, and Jianwei Huang. Harmony: Heterogeneous multi-modal federated learning through disentangled model training. In *Proceedings of the 21st Annual International Conference on Mobile Systems, Applications and Services*, pages 530–543, 2023.

[4] Shuaijun Chen, Omid Tavallaie, Michael Henri Hambali, Seid Miad Zandavi, Hamed Haddadi, Song Guo, and Albert Y Zomaya. Boosting client selection of federated learning under device and data heterogeneity. *arXiv preprint arXiv:2310.08147*, 2023.

[5] Kunjal Panchal, Sunav Choudhary, Subrata Mitra, Koyel Mukherjee, Somdeb Sarkhel, Saayan Mitra, and Hui Guan. Flash: concept drift adaptation in federated learning. In *International Conference on Machine Learning*, pages 26931–26962. PMLR, 2023.

[6] Meisam Hejazinia, Dzmitry Huba, Ilias Leontiadis, Kiwan Maeng, Mani Malek, Luca Melis, Ilya Mironov, Milad Nasr, Kaikai Wang, and Carole-Jean Wu. Fel: High capacity learning for recommendation and ranking via federated ensemble learning. *arXiv preprint arXiv:2206.03852*, 2022.

[7] Hao Zhang, Tingting Wu, Siyao Cheng, and Jie Liu. Fedcos: A scene-adaptive enhancement for federated learning. *IEEE Internet of Things Journal*, 10(5):4545–4556, 2022.

[8] Liangqi Yuan, Yunsheng Ma, Lu Su, and Ziran Wang. Peer-to-peer federated continual learning for naturalistic driving action recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5249–5258, 2023.

[9] Ang Li, Jingwei Sun, Pengcheng Li, Yu Pu, Hai Li, and Yiran Chen. Hermes: an efficient federated learning framework for heterogeneous mobile clients. In *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*, pages 420–437, 2021.

[10] Sameer Bibikar, Haris Vikalo, Zhangyang Wang, and Xiaohan Chen. Federated dynamic sparse training: Computing less, communicating less, yet learning better. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 6080–6088, 2022.

[11] Paul Pu Liang, Terrance Liu, Liu Ziyin, Nicholas B Allen, Randy P Auerbach, David Brent, Ruslan Salakhutdinov, and Louis-Philippe Morency. Think locally, act globally: Federated learning with local and global representations. *arXiv preprint arXiv:2001.01523*, 2020.

[12] Hongyi Wang, Mikhail Yurochkin, Yuekai Sun, Dimitris Papailiopoulos, and Yasaman Khazaeni. Federated learning with matched averaging. *arXiv preprint arXiv:2002.06440*, 2020.

[13] Erich Elsen, Marat Dukhan, Trevor Gale, and Karen Simonyan. Fast sparse convnets. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 14629–14638, 2020.

[14] Zichen Ma, Yu Lu, Wenye Li, Jinfeng Yi, and Shuguang Cui. Pfedatt: Attention-based personalized federated learning on heterogeneous clients. In *Asian Conference on Machine Learning*, pages 1253–1268. PMLR, 2021.

[15] Canh T Dinh, Nguyen Tran, and Josh Nguyen. Personalized federated learning with moreau envelopes. *Advances in neural information processing systems*, 33:21394–21405, 2020.

[16] Ang Li, Jingwei Sun, Xiao Zeng, Mi Zhang, Hai Li, and Yiran Chen. Fedmask: Joint computation and communication-efficient personalized federated learning via heterogeneous masking. In *Proceedings of the 19th ACM Conference on Embedded Networked Sensor Systems*, pages 42–55, 2021.

[17] Xiangyi Chen, Guangjie Han, Yuanguo Bi, Zimeng Yuan, Mahesh K Marina, Yufei Liu, and Hai Zhao. Traffic prediction-assisted federated deep reinforcement learning for service migration in digital twins-enabled mec networks. *IEEE Journal on Selected Areas in Communications*, 2023.

[18] Jingwei Sun, Ang Li, Lin Duan, Samiul Alam, Xuliang Deng, Xin Guo, Haiming Wang, Maria Gorlatova, Mi Zhang, Hai Li, et al. Fedsea: A semi-asynchronous federated learning framework for extremely heterogeneous devices. In *Sensys*, pages 106–119, 2022.

[19] Abdulrahman Alabbasi, Milad Ganjalizadeh, Konstantinos Vandikas, and Marina Petrova. On cascaded federated learning for multi-tier predictive models. In *2021 IEEE International Conference on Communications Workshops (ICC Workshops)*, pages 1–7. IEEE, 2021.

[20] Mingkun Yang, Ran Zhu, Qing Wang, and Jie Yang. Fedtrans: Client-transparent utility estimation for robust federated learning. In *The Twelfth International Conference on Learning Representations*, 2023.

[21] Jinhyun So, Kevin Hsieh, Behnaz Arzani, Shadi Noghabi, Salman Avestimehr, and Ranveer Chandra. Fedspace: An efficient federated learning framework at satellites and ground stations. *arXiv preprint arXiv:2202.01267*, 2022.

[22] Shenghong Dai, SM Iftekharul Alam, Ravikumar Balakrishnan, Kangwook Lee, Suman Banerjee, and Nageen Himayat. Online federated learning based object detection across autonomous vehicles in a virtual world. In *2023 IEEE 20th Consumer Communications & Networking Conference (CCNC)*, pages 919–920. IEEE, 2023.

[23] Jaemin Shin, Yuanchun Li, Yunxin Liu, and Sung-Ju Lee. Fedbalancer: Data and pace control for efficient federated learning on heterogeneous clients. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services*, pages 436–449, 2022.

[24] Pytorch official documentation. https://pytorch.org/.

[25] Zhenyu Wang and Shahriar Nirjon. Characterizing disparity between edge models and high-accuracy base models for vision tasks. *arXiv preprint arXiv:2407.10016*, 2024.

[26] Yong Liu and Xin Yao. Ensemble learning via negative correlation. *Neural networks*, 12(10):1399–1404, 1999.

[27] Wei Wen, Chunpeng Wu, Yandan Wang, Yiran Chen, and Hai Li. Learning structured sparsity in deep neural networks. *Advances in neural information processing systems*, 29, 2016.

[28] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.

[29] Simonyan Karen. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv: 1409.1556*, 2014.

[30] Patrick Helber, Benjamin Bischke, Andreas Dengel, and Damian Borth. Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens.*, 12(7):2217–2226, 2019.

[31] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.

[32] Luke N Darlow, Elliot J Crowley, Antreas Antoniou, and Amos J Storkey. Cinic-10 is not imagenet or cifar-10. *arXiv preprint arXiv:1810.03505*, 2018.

[33] Accubattery. https://accubatteryapp.com/.

[34] Dan Alistarh, Demjan Grubic, Jerry Li, Ryota Tomioka, and Milan Vojnovic. Qsgd: Communication-efficient sgd via gradient quantization and encoding. *NeurIPS*, 30, 2017.

[35] Jeremy Bernstein, Yu-Xiang Wang, Kamyar Azizzadenesheli, and Animashree Anandkumar. signsgd: Compressed optimisation for non-convex problems. In *International Conference on Machine Learning*, pages 560–569. PMLR, 2018.

[36] Frank Seide, Hao Fu, Jasha Droppo, Gang Li, and Dong Yu. 1-bit stochastic gradient descent and its application to data-parallel distributed training of speech dnns. In *Interspeech*, volume 2014, pages 1058–1062. Singapore, 2014.

[37] Alham Fikri Aji and Kenneth Heafield. Sparse communication for distributed gradient descent. *arXiv preprint arXiv:1704.05021*, 2017.

[38] Nikita Ivkin, Daniel Rothchild, Enayat Ullah, Ion Stoica, Raman Arora, et al. Communication-efficient distributed sgd with sketching. *Advances in Neural Information Processing Systems*, 32, 2019.

[39] Andrew Hard, Kanishka Rao, Rajiv Mathews, Swaroop Ramaswamy, Françoise Beaufays, Sean Augenstein, Hubert Eichner, Chloé Kiddon, and Daniel Ramage. Federated learning for mobile keyboard prediction. *arXiv preprint arXiv:1811.03604*, 2018.

[40] Yuang Jiang, Shiqiang Wang, Victor Valls, Bong Jun Ko, Wei-Han Lee, Kin K Leung, and Leandros Tassiulas. Model pruning enables efficient federated learning on edge devices. *IEEE Transactions on Neural Networks and Learning Systems*, 2022.

[41] JaeYeon Park, Kichang Lee, Sungmin Lee, Mi Zhang, and JeongGil Ko. Attfl: A personalized federated learning framework for time-series mobile and embedded sensor data processing. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 7(3):1–31, 2023.

Authorized licensed use limited to: Chung-ang Univ. Downloaded on January 21,2026 at 03:29:09 UTC from IEEE Xplore. Restrictions apply.